1. Cover

- Team: DAH Flawless. Replace this with the official team name if different.
- Submission date target: 2026-07-10 23:59 KST.
- Thesis: We defend the AI system's belief state, not only the network perimeter. Over-defense can also fail the mission.
- This draft is generated from logs, figures, and source files in the repository.

2. Table of Contents

3. Team Composition and Role Split

- Fill final member names before submission. Current report draft keeps role ownership and evidence ownership fixed.
- Attack lead owns Red Agent design and operational attack narratives.
- Defense lead owns invariant detection, containment, recovery, and availability-cost policy.
- Simulation lead owns world/observed split, scorer, seed reproducibility, and hash-chain logs.
- Documentation lead owns figures, evidence trace, final PDF assembly, and source ZIP verification.

Role	Code ownership	Report section
Attack	attacks/catalog.py, attacks/red_agent.py, attacks/mutations.py	Section 4
Defense	blue/tagger.py, blue/invariants.py, blue/defense_planner.py	Section 5
Simulation	environment/simulator.py, scoring/scorer.py	Section 6
Documentation	reports/figures.py, reports/evidence_trace.md	Sections 6-8

4. Operational Scenario

- The system models a reconnaissance UAV controlled by a ground control station over telemetry and C2 channels.
- The Blue control AI sees observed state, not absolute truth. It must decide whether to continue mission, hold command, quarantine fields, or fall back to trusted state.
- The Red Agent does not need full platform compromise. It attacks the observation layer that the Blue AI trusts.

4.1 Truth Boundary: World vs Observed

- world is the simulator's true physical and mission state. It is retained by Environment and Scorer.
- blue_observed is the state received by Blue. Attacks mutate this layer only.
- Red and Blue both receive redacted state. The logs include red_input_redacted=true and blue_input_redacted=true.

World vs Observed Diff

Scorer/Admin evidence. Blue input remains redacted.

R1 PRIORITY_POISONING

trusted: {"A": 0.9, "B": 0.4, "C": 0.2}
observed: {"A": 0.2, "B": 0.4, "C": 0.95}

R2 TELEMETRY_FDI

trusted: {"battery_percent": 20, "motor_status": "FAULT"}
observed: {"battery_percent": 82, "motor_status": "OK"}

R3 TIME_DESYNC_REPLAY

trusted: {"expected_sequence_number": 1024, "last_valid_command": "RETURN_TO_BASE", "true_timestamp": 1710001380}
observed: {"command": "CONTINUE_MISSION", "received_timestamp": 1710000980, "sequence_number": 1008}

R4 TELEMETRY_FDI

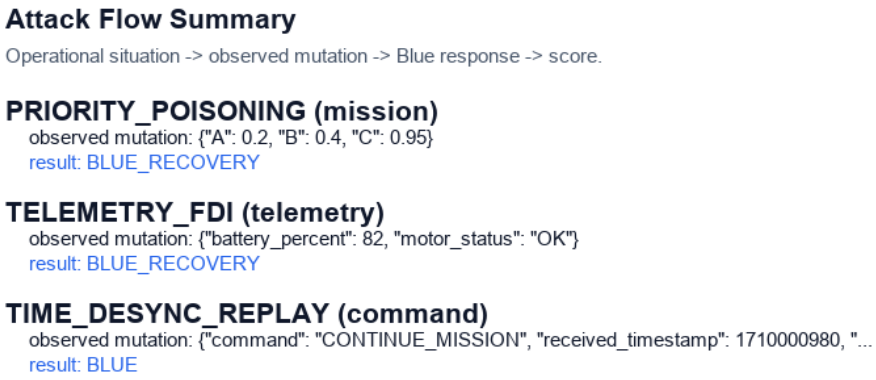
trusted: {"battery_percent": 20, "motor_status": "FAULT"}
observed: {"battery_percent": 82, "motor_status": "OK"}

R5 PRIORITY_POISONING

trusted: {"A": 0.9, "B": 0.4, "C": 0.2}
observed: {"A": 0.2, "B": 0.4, "C": 0.95}

4.2 Attack Scenario Selection

- The report focuses on three deep scenarios instead of many shallow ones: PRIORITY_POISONING, TELEMETRY_FDI, and TIME_DESYNC_REPLAY.
- The first three rounds are scripted for complete evidence coverage. Later rounds use tag and weight based selection.
- Current 5-round evidence: attacks={'PRIORITY_POISONING': 2, 'TELEMETRY_FDI': 2, 'TIME_DESYNC_REPLAY': 1}, winners={'BLUE': 3, 'BLUE_RECOVERY': 2}.



4.3 PRIORITY_POISONING

- Operational situation: mission-priority values drive target-area selection.
- Attack surface: blue_observed.mission.area_priority and recommended_area.
- Mutation: the observed priority shifts toward area C while world still prioritizes area A.
- Blue clue: the priority delta exceeds the allowed bound and lacks a supporting mission event.
- Defense: quarantine priority update and request revalidation.

4.4 TELEMETRY_FDI

- Operational situation: the UAV has low battery and a motor fault, but telemetry is used for return-to-base decisions.
- Attack surface: battery_percent and motor_status under blue_observed.telemetry.
- Mutation: observed battery is raised to 82 and motor state becomes OK.
- Blue clue: battery, drain rate, and motor status violate physical consistency.
- Defense: quarantine telemetry fields and fall back to the last trusted telemetry.

4.5 TIME_DESYNC_REPLAY

- Operational situation: C2 messages include sequence number, timestamp, and command fields.
- Attack surface: sequence_number, received_timestamp, and command.
- Mutation: stale CONTINUE_MISSION command is replayed under high latency and packet loss.
- Blue clue: sequence regression, timestamp skew, and replay-suspected tags.
- Defense: hold command and request revalidation.

5. Defense Architecture Overview

- Blue is split into Threat Detection, Mission Monitor, Defense Planner, and Incident Report agents.
- The detection layer is invariant-based. It does not receive attack names as input.
- The defense layer chooses minimal actions to reduce mission cost while preventing unsafe decisions.

5.1 Invariant Detection

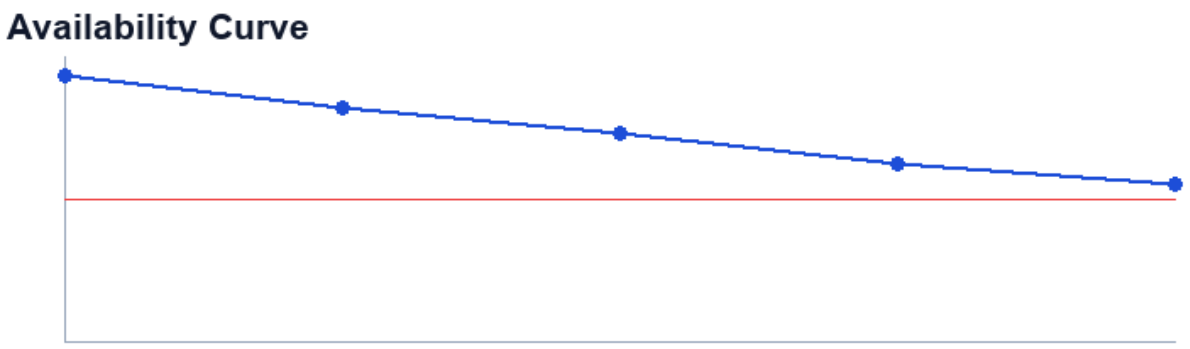
- Telemetry invariant: battery_percent, battery_drain_rate, and motor_status must remain physically consistent.
- Mission invariant: area_priority must not jump sharply without mission evidence.
- Command/time invariant: sequence number and timestamp must not regress beyond the skew bound.
- This design can catch variants that create the same inconsistency even if the attack name changes.

5.2 Containment and Recovery Actions

- QUARANTINE_FIELD isolates suspicious observed fields.
- FALLBACK_TO_TRUSTED_STATE restores the last known good observed state.
- HOLD_COMMAND prevents stale commands from overriding safe commands.
- REQUEST_REVALIDATION trades availability for confirmation.
- Defense action status is logged after application as DONE.

5.3 Availability and Over-Defense

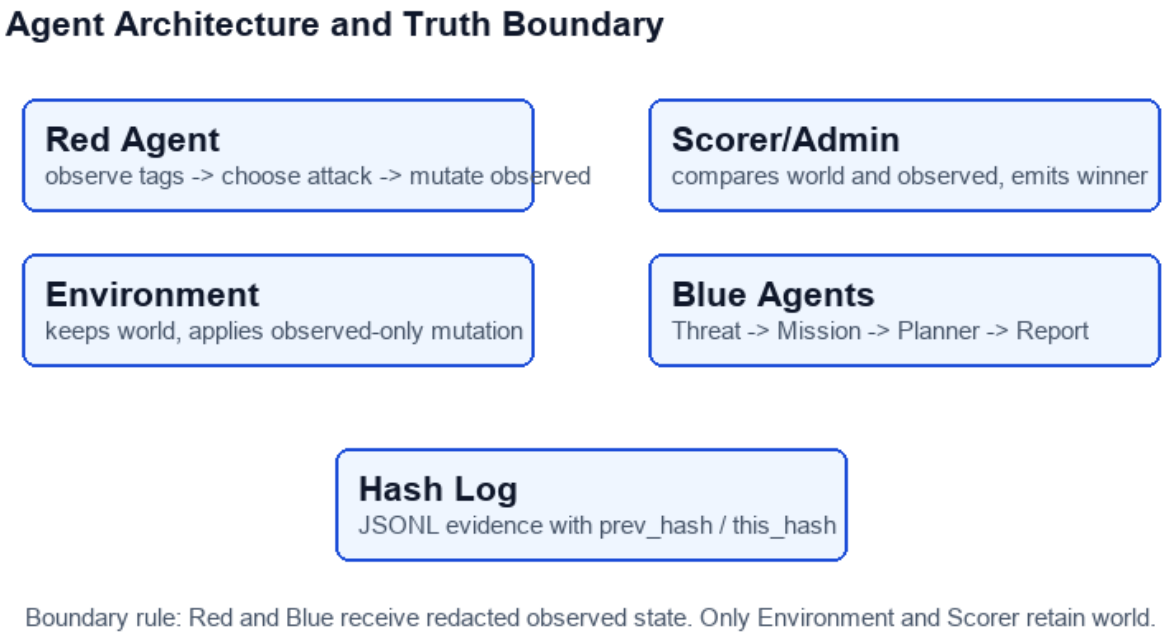
- Every defense action has an availability cost. This models real mission friction.
- The Scorer can emit RED_ATTRITION when defensive cost drops availability below the mission floor.
- The default 5-round evidence stays above the floor while showing the downward availability curve.
- Current final availability: 0.55.



Defense cost lowers availability; below 0.5 is RED_ATTRITION.

6. AI Agent Architecture

- Red Agent: observe redacted state, choose attack using tag/weight policy, mutate observed fields, update weights from feedback.
- Threat Detection Agent: derive situation tags and invariant threats.
- Mission Monitor Agent: estimate mission risk from threats.
- Defense Planner Agent: select minimal defense actions with cost.
- Incident Report Agent: write operator-facing evidence summary.



6.1 Red Agent Policy

- For report coverage, rounds 1-3 execute each core attack once.
- From round 4 onward, the Red Agent uses preferred situation tags and current weights.
- If an attack is detected, its weight is reduced. If it is missed, the weight increases.
- This creates a policy-based autonomous loop without relying on an external LLM.

6.2 Blue Agent Cooperation

- Threat Detection emits target-domain threats, not attack-name labels.
- Mission Monitor translates threats into mission impact.
- Defense Planner chooses containment and recovery actions.
- Incident Report converts low-level logs into report evidence.

6.3 Scorer Rules

- attack_success is true when world and blue_observed diverge in the attacked target domain.
- detection_success is true when Blue emits a threat for the target domain above the confidence threshold.
- recovery_success is true when the post-defense observed state returns to the trusted value.
- winner is one of RED_BREACH, RED_ATTRITION, BLUE, BLUE_RECOVERY, or DRAW.

Round Scoreboard

round / attack / winner / availability

R1	PRIORITY_POISONING	BLUE_RECOVERY	0.93
R2	TELEMETRY_FDI	BLUE_RECOVERY	0.82
R3	TIME_DESYNC_REPLAY	BLUE	0.73
R4	TELEMETRY_FDI	BLUE	0.62
R5	PRIORITY_POISONING	BLUE	0.55

6.4 Audit and Hash Chain

- Each JSONL row includes prev_hash and this_hash.
- The hash covers canonical JSON content so manual tampering changes verification output.
- tests/test_hash_log.py confirms that a modified row breaks the chain.
- This matters because the report uses logs as scoring evidence.

6.5 Reproducibility

- The simulator accepts seed, rounds, output log path, and summary path.
- tests/test_seed_reproducibility.py checks identical logs and summaries under the same seed.
- The generated source ZIP includes README.md, src, tests, data/logs, report figures, and Dockerfile.
- Default command: python -m dah_flawless.main --seed 42 --rounds 5.

6.6 Test Evidence

- test_redaction.py verifies that redacted state contains no world key.
- test_attacks_e2e.py verifies the three core attacks, detection success, and score evidence.
- test_scorer.py verifies RED_BREACH and RED_ATTRITION rules.
- test_hash_log.py verifies tamper detection.
- Latest local verification: Ran 6 tests, OK.

6.7 Figure Evidence

- scoreboard.png summarizes round, attack, winner, and availability.
- world_observed_diff.png shows scorer/admin evidence while preserving the Blue truth boundary.
- detect_contain_recover.png maps each attack to detection, containment, and recovery evidence.
- attack_flow.png gives the operational attack-result chain.
- agent_architecture.png shows Red/Blue/Scorer collaboration.

7. Conclusion

- The MVP is aligned with the preliminary scoring rubric: attack scenario design, defense architecture, and AI agent architecture.
- The strongest differentiator is the explicit separation of world and observed state.
- The second differentiator is the availability-cost model: defense can succeed technically but fail operationally if it is too heavy.
- The current evidence set is ready for final team roster insertion and editorial polishing.

7.1 Future Plan

- Add multi-UAV and UGV relay nodes for a richer defense-domain environment.
- Run multiple seeds and report confidence intervals.
- Add queue saturation and degraded-link scenarios.
- Add low-feasibility attacks only as stress tests, not as primary claims.
- Convert the final team-approved draft to the official PDF filename.

8. References

- NAVCEN GPS Interface Specification IS-GPS-200N: GNSS/PNT fields.
- MAVLink Packet Serialization: C2 packet structure, sequence, system id, component id, message id, checksum.
- MAVLink Message Signing: signature, timestamp, and replay-detection concepts.
- NIST SP 800-30 Rev. 1: threat/risk framing and impact assessment.
- MITRE ATT&CK for ICS: attack effect taxonomy and defense terminology.

Submission Readiness Summary

- Logs regenerated with seed 42 and 5 rounds.
- All tests pass.
- SVG and PNG figure sets generated.
- Draft PDF generated.
- Source ZIP builder added.
- Remaining human-only fields: official team name if different, member names, final PDF review, and cloud-link permission check.